MScFE 632 — Machine Learning in Finance

GWP1: Comparing seven machine learning methods

This notebook compares seven methods from the course using a stock-month dataset. We use Ridge regression to predict the supplied forward one-month return. A classification tree, Linear Discriminant Analysis (LDA), a Support Vector Machine (SVM), and a neural network predict whether the absolute return exceeds 4%. This label identifies larger moves in either direction; it does not tell us whether to buy or sell. K-means and Principal Component Analysis (PCA) are used to explore the input features.

The aim is to understand what each method does and where it struggles with the same small dataset. We keep the model searches fairly small and reserve the last four years for testing. This is the computational part of GWP1; the examples cover all seven categories.

Run: keep `data/mlfac_gwp1.csv` next to this notebook in its `data` folder, then restart the kernel and run all cells. No download or API key is needed. The package versions used for the saved results appear below.

1. Data and setup

We use the `mlfac_dat.csv` file supplied with Module 3. The local file has 50 anonymous stock IDs and 11,284 monthly observations from 2000 to 2018. The accompanying CSV keeps the six features, the return, the date, and the stock ID. No rows were removed when making this copy.

Two details matter here. The source documentation defines `R1M_Usd` as a **forward one-month total return**, including dividends, so we do not shift it again. The features have already been uniformized separately at each date. A value of 0.8 is a relative position in the source cross-section, not an original dollar amount or ratio. The 50-stock course file is a subset of that source universe. Coqueret and Guida, data preparation.

Input	Meaning before the source transformation
`Mkt_Cap_12M_Usd`	Average market capitalization over 12 months
`Pb`	Price-to-book ratio
`Vol1Y_Usd`	One-year return volatility
`Return_On_Capital`	Return on capital

Input	Meaning before the source transformation
`Roe`	Return on equity
`Pe`	Price-to-earnings ratio

In particular, `Vol1Y_Usd` measures return volatility, rather than trading volume. These definitions follow the source data dictionary. Stock IDs and dates are used for checks and splitting, not as model inputs. Other forward-return columns are excluded.

In [1]:
```
from pathlib import Path
import hashlib
import platform
import warnings
from importlib.metadata import version

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.base import clone
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.dummy import DummyClassifier, DummyRegressor
from sklearn.exceptions import ConvergenceWarning
from sklearn.linear_model import Ridge
from sklearn.metrics import (
    accuracy_score, balanced_accuracy_score, confusion_matrix,
    ConfusionMatrixDisplay, f1_score, mean_absolute_error,
    r2_score, root_mean_squared_error, roc_auc_score, roc_curve,
    silhouette_score,
)
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, plot_tree

SEED = 42
FEATURES = ['Mkt_Cap_12M_Usd', 'Pb', 'Vol1Y_Usd',
            'Return_On_Capital', 'Roe', 'Pe']
LABELS = ['Market cap', 'P/B', 'Volatility', 'ROC', 'ROE', 'P/E']
THRESHOLD = 0.04
warnings.simplefilter('error', ConvergenceWarning)
plt.rcParams.update({'figure.dpi': 110, 'axes.spines.top': False,
                     'axes.spines.right': False, 'font.size': 10})
pd.set_option('display.precision', 4)
pd.set_option('display.max_colwidth', None)

display(pd.Series({
    'Python': platform.python_version(),
    **{p: version(p) for p in ['numpy', 'pandas', 'scikit-learn', 'matplo
}, name='Version').to_frame())
```

	Version
Python	3.14.0
numpy	2.4.1
pandas	2.3.3
scikit-learn	1.9.0
matplotlib	3.10.8

In [2]:
```
# Support running from this folder or from the repository root.
data_paths = [Path('data/mlfac_gwp1.csv'),
              Path('modules/5_ML_IN_FINANCE/GWP1/data/mlfac_gwp1.csv')]
data_path = next((p for p in data_paths if p.is_file()), None)
if data_path is None:
    raise FileNotFoundError('Keep data/mlfac_gwp1.csv in the GWP1 folder.

expected_hash = 'e4a5242194d9609920d9dff2997e25791782814c1a49d48a319f3c87
assert hashlib.sha256(data_path.read_bytes()).hexdigest() == expected_has
data = pd.read_csv(data_path, parse_dates=['date'])
assert data.shape == (11284, 9)
assert not data.isna().any().any()
assert not data.duplicated(['stock_id', 'date']).any()
assert np.isfinite(data[FEATURES + ['R1M_Usd']].to_numpy()).all()
assert data['date'].dt.is_month_end.all()
assert data[FEATURES].ge(0).all().all() and data[FEATURES].le(1).all().al

data = data.sort_values(['date', 'stock_id']).reset_index(drop=True)
data['large_move'] = (data['R1M_Usd'].abs() > THRESHOLD).astype(int)
display(data.head())
print(f"Rows: {len(data):,}; stocks: {data.stock_id.nunique()}; "
      f"months: {data.date.nunique()}; missing values: {data.isna().sum()
```

	stock_id	date	Mkt_Cap_12M_Usd	Pb	Vol1Y_Usd	Return_On_Capital	Roe	
0	1	2000-01-31	0.52	0.21	0.77	0.08	0.02	0
1	2	2000-01-31	0.49	0.85	0.91	0.61	0.68	0
2	3	2000-01-31	0.11	0.19	0.22	0.14	0.30	0
3	4	2000-01-31	0.40	0.26	0.21	0.37	0.25	0
4	5	2000-01-31	0.63	0.56	0.76	0.05	0.21	0

```
Rows: 11,284; stocks: 50; months: 228; missing values: 0
```

2. Targets and a time-based comparison

The regression target is r_{t+1}, the supplied forward monthly total return. Following the tree example in Module 3, the classification target is

$$y_t = \mathbf{1}\{|r_{t+1}| > 0.04\}.$$

Class 1 therefore includes both gains above 4% and losses below −4%. The cutoff is fixed before fitting. We use it to stay close to the lesson, rather than because 4% has been shown to be an optimal trading rule.

The development period ends in November 2014. December 2014 is left out, and January 2015–December 2018 is the holdout period. Within development, we use three expanding time splits, each with 36 validation months and a one-month gap. Splitting the unique dates first keeps all stocks from the same month together. We assume a forward one-month label is available by the end of the following month.

All scaling is fitted inside the supervised pipelines, separately for each training fold. Ridge is selected by validation mean squared error; the four classifiers are selected by validation ROC AUC. AUC measures ranking across thresholds, while the later confusion matrices use each model's default classification rule. The grids and the split are set here, before looking at holdout performance. See the TimeSeriesSplit documentation for the expanding-window scheme.

In [3]:
```
dev = data.loc[data.date <= '2014-11-30'].reset_index(drop=True)
test = data.loc[data.date >= '2015-01-01'].reset_index(drop=True)
gap = data.loc[(data.date > '2014-11-30') & (data.date < '2015-01-01')]
X_dev, X_test = dev[FEATURES], test[FEATURES]
y_dev, y_test = dev['R1M_Usd'], test['R1M_Usd']
c_dev, c_test = dev['large_move'], test['large_move']

assert len(dev) + len(gap) + len(test) == len(data)
assert set(dev.date).isdisjoint(set(test.date))
assert dev.date.max() + pd.offsets.MonthEnd(1) < test.date.min()
assert set(test.stock_id).issubset(set(dev.stock_id))

months = np.sort(dev.date.unique())
month_splitter = TimeSeriesSplit(n_splits=3, test_size=36, gap=1)
cv_splits, split_rows = [], []
for fold, (train_months, valid_months) in enumerate(month_splitter.split(
    train_idx = np.flatnonzero(dev.date.isin(months[train_months]))
    valid_idx = np.flatnonzero(dev.date.isin(months[valid_months]))
    train_dates, valid_dates = dev.date.iloc[train_idx], dev.date.iloc[va
    assert set(train_dates).isdisjoint(set(valid_dates))
    assert train_dates.max() + pd.offsets.MonthEnd(1) < valid_dates.min()
    assert c_dev.iloc[train_idx].nunique() == c_dev.iloc[valid_idx].nuniq
    cv_splits.append((train_idx, valid_idx))
    split_rows.append({
        'Fold': fold, 'Training start': train_dates.min().date(),
        'Training end': train_dates.max().date(),
        'Validation start': valid_dates.min().date(),
        'Validation end': valid_dates.max().date(),
        'Training rows': len(train_idx), 'Validation rows': len(valid_idx
    })

display(pd.DataFrame([
    {'Period': name, 'Start': frame.date.min().date(), 'End': frame.date.
     'Rows': len(frame), 'Months': frame.date.nunique()}
    for name, frame in [('Development', dev), ('Gap', gap), ('Holdout', t
]).set_index('Period'))
display(pd.DataFrame(split_rows).set_index('Fold'))
```

```
def run_search(estimator, grid, target, scoring):
    search = GridSearchCV(estimator, grid, cv=cv_splits, scoring=scoring,
                          refit=True, n_jobs=1, error_score='raise')
    search.fit(X_dev, target)
    return search

def show_search(search):
    results = pd.DataFrame(search.cv_results_)
    display(results[['params', 'mean_test_score', 'std_test_score']]
            .sort_values('mean_test_score', ascending=False)
            .rename(columns={'mean_test_score': 'Mean CV score',
                             'std_test_score': 'SD across folds'})
            .reset_index(drop=True))
    print('Selected:', search.best_params_)
```

Period	Start	End	Rows	Months
Development	2000-01-31	2014-11-30	8858	179
Gap	2014-12-31	2014-12-31	50	1
Holdout	2015-01-31	2018-12-31	2376	48

Fold	Training start	Training end	Validation start	Validation end	Training rows	Validation rows
1	2000-01-31	2005-10-31	2005-12-31	2008-11-30	3408	1800
2	2000-01-31	2008-10-31	2008-12-31	2011-11-30	5208	1800
3	2000-01-31	2011-10-31	2011-12-31	2014-11-30	7008	1800

The same stock can appear on both sides of a time split. This experiment asks whether a fitted relationship carries into later months for this stock sample; it does not test generalization to new companies. The gap reduces overlap from forward labels, but it cannot repair unknown reporting delays or survivorship bias in a preprocessed dataset.

Before fitting, we inspect only the development sample. Returns are kept as supplied, including extreme observations. This makes squared-error results sensitive to unusually large returns, but avoids choosing a cleaning rule after seeing the test results.

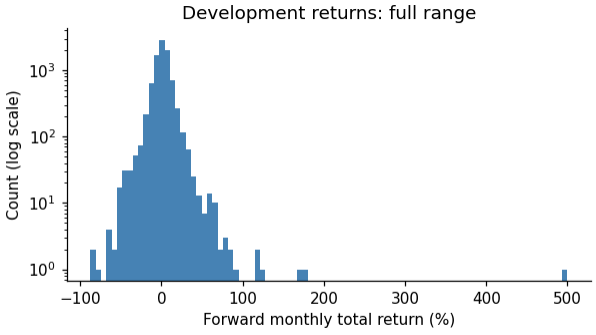
The largest development return is 500% in one month. We could not independently check that observation. Keeping it unchanged is transparent, but it is a limitation of this teaching sample and a reason to report MAE alongside RMSE.

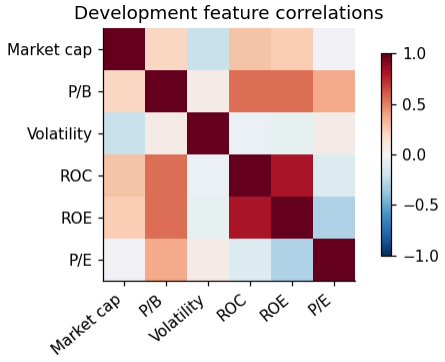
In [4]:
```
display(dev[FEATURES + ['R1M_Usd']].describe().T[['mean', 'std', 'min', '
print(f'Development large-move share: {c_dev.mean():.2%}')
fig, axes = plt.subplots(1, 2, figsize=(11, 3.5))
```

```
axes[0].hist(100 * y_dev, bins=90, color='steelblue', log=True)
axes[0].set(xlabel='Forward monthly total return (%)', ylabel='Count (log
            title='Development returns: full range')
corr = X_dev.corr()
im = axes[1].imshow(corr, vmin=-1, vmax=1, cmap='RdBu_r')
axes[1].set(xticks=range(6), yticks=range(6), xticklabels=LABELS,
            yticklabels=LABELS, title='Development feature correlations')
plt.setp(axes[1].get_xticklabels(), rotation=40, ha='right')
fig.colorbar(im, ax=axes[1], shrink=0.8)
fig.tight_layout()
plt.show()
```

	mean	std	min	50%	max
Mkt_Cap_12M_Usd	0.5017	0.2913	0.010	0.480	1.0
Pb	0.4698	0.2902	0.010	0.460	1.0
Vol1Y_Usd	0.4756	0.2926	0.010	0.440	1.0
Return_On_Capital	0.5054	0.2807	0.010	0.510	1.0
Roe	0.4947	0.2798	0.010	0.500	1.0
Pe	0.4908	0.2663	0.010	0.500	1.0
R1M_Usd	0.0139	0.1335	-0.875	0.013	5.0

```
Development large-move share: 60.74%
```





3. Ridge regression — category 1

Ridge is a linear regression with a penalty on the coefficient sizes:

$$\min_{\beta_0,\beta} \sum_i (r_i - \beta_0 - x_i^\top \beta)^2 + \alpha \sum_j \beta_j^2.$$

The inputs are the six standardized features and the output is a predicted monthly return. Increasing α shrinks coefficients toward zero; the intercept is not penalized. This is useful when related predictors produce unstable coefficients. The cost is extra bias, and a linear model still cannot capture all possible interactions.

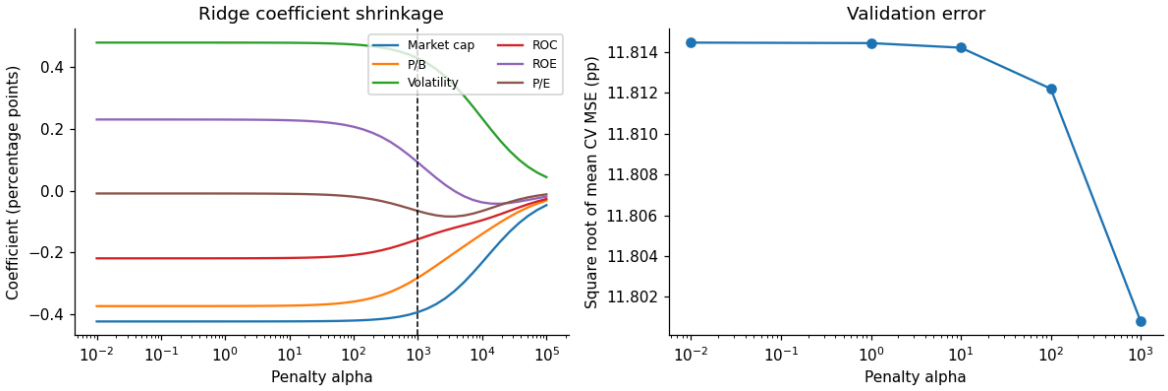
We compare five penalty values using mean squared error on the time folds. The search table reports negative MSE, so a larger score is better. The coefficient path below shows how the fitted relationship changes with the penalty. Coefficients are in return units per one development-sample standard deviation, rather than per original dollar or ratio unit.

In [5]:
```
ridge = run_search(
    Pipeline([('scale', StandardScaler()), ('model', Ridge())]),
    {'model__alpha': [0.01, 1, 10, 100, 1000]}, y_dev, 'neg_mean_squared_
show_search(ridge)

alphas = np.logspace(-2, 5, 50)
ridge_path = []
for alpha in alphas:
    fitted = clone(ridge.best_estimator_).set_params(model__alpha=alpha).
    ridge_path.append(fitted.named_steps['model'].coef_)
ridge_path = np.asarray(ridge_path)
fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
for j, label in enumerate(LABELS):
    axes[0].semilogx(alphas, 100 * ridge_path[:, j], label=label)
axes[0].axvline(ridge.best_params_['model__alpha'], color='black', ls='--
axes[0].set(xlabel='Penalty alpha', ylabel='Coefficient (percentage point
            title='Ridge coefficient shrinkage')
axes[0].legend(fontsize=8, ncol=2)
ridge_cv = pd.DataFrame(ridge.cv_results_)
axes[1].semilogx(ridge_cv.param_model__alpha.astype(float),
                 100 * np.sqrt(-ridge_cv.mean_test_score), marker='o')
axes[1].set(xlabel='Penalty alpha', ylabel='Square root of mean CV MSE (p
            title='Validation error')
fig.tight_layout()
plt.show()
```

	params	Mean CV score	SD across folds
0	{'model__alpha': 1000}	-0.0139	0.0050
1	{'model__alpha': 100}	-0.0140	0.0050
2	{'model__alpha': 10}	-0.0140	0.0051
3	{'model__alpha': 1}	-0.0140	0.0051
4	{'model__alpha': 0.01}	-0.0140	0.0051

```
Selected: {'model__alpha': 1000}
```



The search selects $\alpha = 1000$, the largest value in the grid. The validation error changes very little across the five candidates: the square root of mean CV MSE stays close to 11.8 percentage points. Among these candidates, stronger shrinkage gives a slightly lower validation error, but the improvement is small. The path also shows that individual coefficients need not shrink monotonically when features are correlated. This is a bounded search, not evidence that 1000 is the best possible penalty.

4. K-means clustering — category 2

K-means groups observations by their feature distances. It minimizes the within-cluster sum of squares:

$$\sum_{k=1}^{K} \sum_{i:c_i=k} \|z_i - \mu_k\|^2,$$

where z_i is a standardized feature vector and μ_k is a cluster center. It takes features without a return label and produces a cluster assignment and centers. This makes it a useful way to describe common stock profiles. It also assumes that Euclidean distance is meaningful; scaling, outliers, and the chosen number of groups can change the result.

We compare $K = 2, 3, 4, 5$ using development data only. The silhouette score compares within-cluster closeness with separation from other clusters. We select the highest score from this small range, using a fixed sample of 2,500 rows for the calculation. Twenty initializations reduce sensitivity to one random starting point. These are stock-month clusters: a stock can move between groups over time.

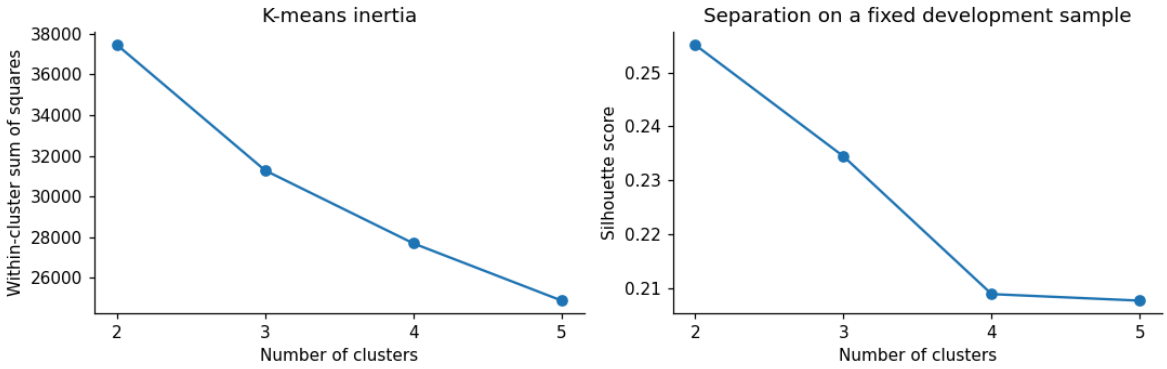
In [6]:
```
unsupervised_scaler = StandardScaler().fit(X_dev)
Z_dev = unsupervised_scaler.transform(X_dev)
cluster_models, cluster_rows = {}, []
for k in range(2, 6):
    km = KMeans(n_clusters=k, n_init=20, random_state=SEED).fit(Z_dev)
    cluster_models[k] = km
    cluster_rows.append({'K': k, 'Inertia': km.inertia_,
                         'Silhouette': silhouette_score(
                             Z_dev, km.labels_, sample_size=2500, random_
cluster_scores = pd.DataFrame(cluster_rows).set_index('K')
best_k = int(cluster_scores.Silhouette.idxmax())
kmeans = cluster_models[best_k]
display(cluster_scores)
```

```
print('Selected K:', best_k)

fig, axes = plt.subplots(1, 2, figsize=(10, 3.3))
axes[0].plot(cluster_scores.index, cluster_scores.Inertia, marker='o')
axes[0].set(xlabel='Number of clusters', ylabel='Within-cluster sum of sq
            title='K-means inertia', xticks=cluster_scores.index)
axes[1].plot(cluster_scores.index, cluster_scores.Silhouette, marker='o')
axes[1].set(xlabel='Number of clusters', ylabel='Silhouette score',
            title='Separation on a fixed development sample', xticks=clus
fig.tight_layout()
plt.show()
```

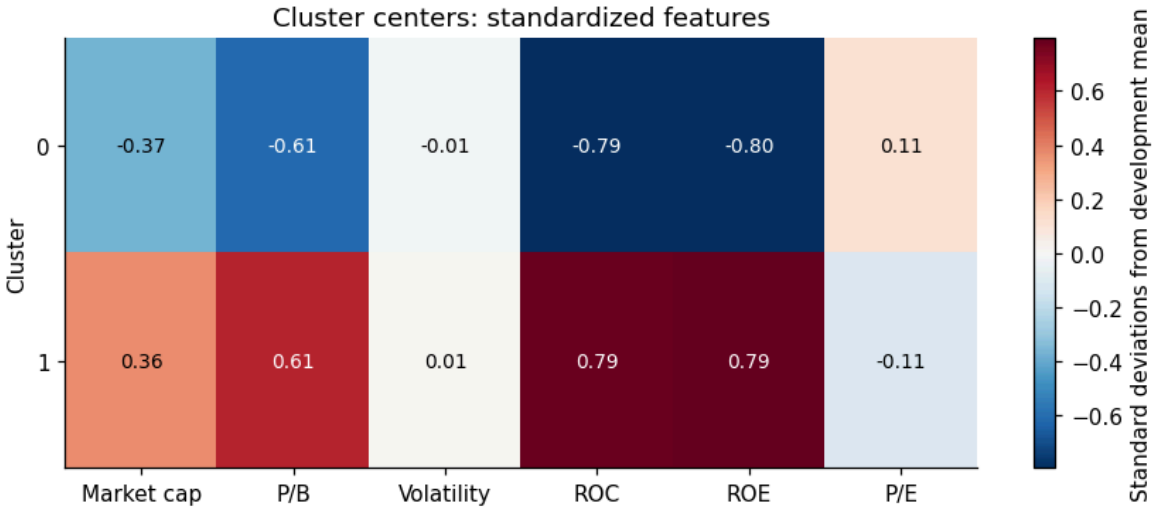
K	Inertia	Silhouette
2	37454.0168	0.2551
3	31280.6611	0.2344
4	27696.6914	0.2088
5	24884.2162	0.2076

```
Selected K: 2
```



In [7]:
```
centers = pd.DataFrame(kmeans.cluster_centers_, columns=LABELS)
centers.index.name = 'Cluster'
sizes = pd.Series(kmeans.labels_).value_counts().sort_index().rename('Sto
display(sizes.to_frame())
fig, ax = plt.subplots(figsize=(8, 3.5))
limit = np.abs(centers.to_numpy()).max()
im = ax.imshow(centers, cmap='RdBu_r', vmin=-limit, vmax=limit, aspect='a
ax.set(xticks=range(6), xticklabels=LABELS, yticks=range(best_k),
       yticklabels=centers.index, ylabel='Cluster', title='Cluster center
for i in range(best_k):
    for j in range(6):
        ax.text(j, i, f'{centers.iloc[i, j]:.2f}', ha='center', va='cente
                color='white' if abs(centers.iloc[i, j]) > 0.55 * limit e
fig.colorbar(im, ax=ax, label='Standard deviations from development mean'
fig.tight_layout()
plt.show()
```

	Stock-months
0	4413
1	4445



Two clusters give the highest sampled silhouette score, 0.255, with 4,413 and 4,445 stock-months. The clearest differences are in profitability and P/B: cluster 1 has higher ROC, ROE, and P/B ranks, while cluster 0 has lower values. Their average volatility ranks are almost the same. The modest silhouette score suggests overlapping profiles rather than two sharply separated types of stock. Cluster numbers are arbitrary labels, not a ranking of investment quality.

5. Principal Component Analysis — category 3

PCA replaces correlated inputs with orthogonal combinations. For the standardized data covariance matrix S, each direction satisfies $Sv_j = \lambda_j v_j$, and the score of an observation is $z_i^\top v_j$. The explained-variance share is $\lambda_j / \sum_l \lambda_l$.

The input is the feature matrix and the outputs are component scores, directions, and explained variance. PCA can reduce redundancy, although components are less directly interpretable than the original features. It maximizes feature variance, not return predictability. We fit it on development data, inspect how many components retain 90% of variance, and plot the first two scores. PCA is an exploration here; it is not an additional input to the classifiers.

In [8]:
```
pca = PCA().fit(Z_dev)
pc_scores = pca.transform(Z_dev)
pc_names = [f'PC{i}' for i in range(1, 7)]
variance = pd.DataFrame({'Explained share': pca.explained_variance_ratio_
                         'Cumulative share': pca.explained_variance_ratio_
                        index=pc_names)
n90 = int(np.searchsorted(variance['Cumulative share'].to_numpy(), 0.90)
display(variance)
print('Components needed for 90% of development variance:', n90)

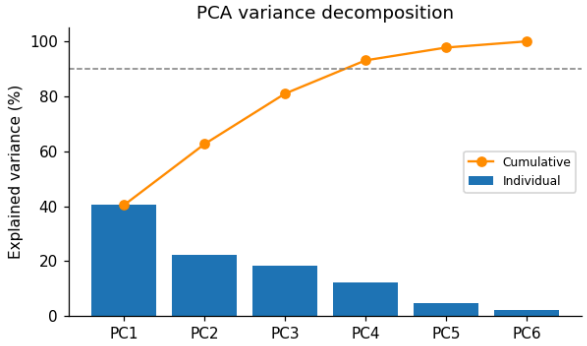
fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))
axes[0].bar(pc_names, 100 * variance['Explained share'], label='Individua
axes[0].plot(pc_names, 100 * variance['Cumulative share'], marker='o', co
             label='Cumulative')
axes[0].axhline(90, ls='--', color='gray', lw=1)
axes[0].set(ylabel='Explained variance (%)', title='PCA variance decompos
axes[0].legend(fontsize=8)
sample = np.random.default_rng(SEED).choice(len(Z_dev), size=2500, replac
```

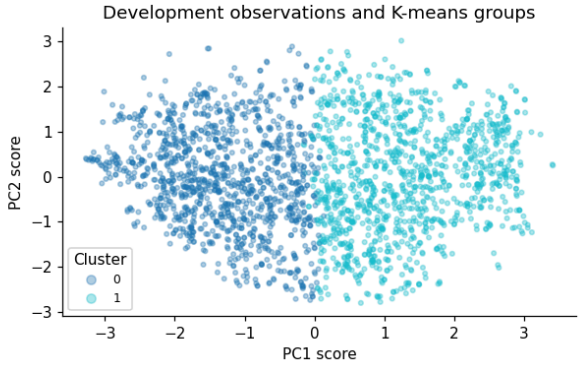
```
scatter = axes[1].scatter(pc_scores[sample, 0], pc_scores[sample, 1],
                          c=kmeans.labels_[sample], cmap='tab10', s=9, al
axes[1].set(xlabel='PC1 score', ylabel='PC2 score', title='Development ob
axes[1].legend(*scatter.legend_elements(), title='Cluster', fontsize=8)
fig.tight_layout()
plt.show()

# Directions are eigenvector coefficients, not correlations with componen
directions = pd.DataFrame(pca.components_[:2].T, index=LABELS, columns=['
display(directions)
```

	Explained share	Cumulative share
PC1	0.4040	0.4040
PC2	0.2220	0.6260
PC3	0.1837	0.8097
PC4	0.1211	0.9308
PC5	0.0467	0.9775
PC6	0.0225	1.0000

```
Components needed for 90% of development variance: 4
```





	PC1	PC2
Market cap	0.2976	-0.1486
P/B	0.4738	0.4885
Volatility	-0.0819	0.3487
ROC	0.5812	-0.0340
ROE	0.5820	-0.1589
P/E	-0.0604	0.7690

PC1 explains 40.4% of development variance and the first two components explain 62.6%. Four components are needed to exceed 90%, reaching 93.1%. PC1 has its largest weights on ROC, ROE, and P/B, which agrees with the cluster-center differences. PC2 gives more weight to P/E and P/B. The two-dimensional plot is useful for seeing the broad groups, but it leaves out 37.4% of the feature variance and says nothing by itself about future returns.

6. Classification tree — category 4

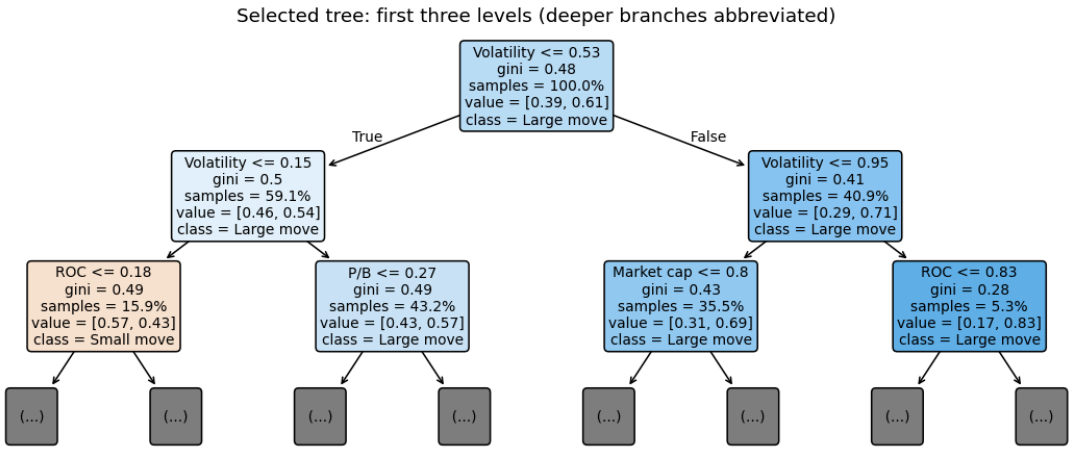
A classification tree repeatedly splits a feature at a threshold. With Gini impurity, a node's impurity is $G = 1 - \sum_k p_k^2$. Each split is chosen to reduce the sample-weighted child impurity. A leaf's class frequencies provide estimated probabilities.

The six ranked features are the inputs and the large-move class is the output. Trees are easy to follow and can capture interactions without scaling. Deep trees can also fit noise. We tune maximum depth and minimum leaf size to keep this example fairly small. A one-variable threshold has a simple meaning, but a split on a ranked value is not a dollar threshold.

In [9]:
```
tree = run_search(DecisionTreeClassifier(random_state=SEED),
                  {'max_depth': [2, 3, 4], 'min_samples_leaf': [50, 100]}
                  c_dev, 'roc_auc')
show_search(tree)
fig, ax = plt.subplots(figsize=(13, 5))
plot_tree(tree.best_estimator_, max_depth=2, feature_names=LABELS,
          class_names=['Small move', 'Large move'], filled=True, rounded=
          proportion=True, precision=2, fontsize=9, ax=ax)
ax.set_title('Selected tree: first three levels (deeper branches abbrevia
plt.show()
display(pd.Series(tree.best_estimator_.feature_importances_, index=LABELS
                  name='Gini importance').sort_values(ascending=False).to
```

	params	Mean CV score	SD across folds
0	{'max_depth': 3, 'min_samples_leaf': 50}	0.6084	0.0063
1	{'max_depth': 3, 'min_samples_leaf': 100}	0.6077	0.0067
2	{'max_depth': 4, 'min_samples_leaf': 100}	0.6067	0.0054
3	{'max_depth': 4, 'min_samples_leaf': 50}	0.6049	0.0062
4	{'max_depth': 2, 'min_samples_leaf': 50}	0.6017	0.0034
5	{'max_depth': 2, 'min_samples_leaf': 100}	0.6017	0.0034

```
Selected: {'max_depth': 3, 'min_samples_leaf': 50}
```



	Gini importance
Volatility	0.9138
ROC	0.0336
P/B	0.0318
Market cap	0.0209
ROE	0.0000
P/E	0.0000

The selected tree has depth 3 and a minimum leaf size of 50, with mean validation AUC of 0.6084. Its first split is on volatility at a supplied rank of about 0.53. Volatility also accounts for 91.4% of its total impurity-based importance. This is a sensible relationship for an absolute-return target, although it does not establish causality. The smaller importance values for the remaining features describe this fitted tree, not their value in every possible model.

7. Linear Discriminant Analysis — category 5

LDA models the feature distribution within each class as Gaussian, with class-specific means and a shared covariance matrix. It chooses the larger discriminant score:

$$\delta_k(x) = x^\top \Sigma^{-1} \mu_k - \tfrac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k.$$

The inputs are features and training labels; the outputs are class scores, probabilities, and predictions. Sharing the covariance matrix gives a linear boundary and keeps the model relatively simple. However, ranked financial features need not be Gaussian, and the two classes may have different covariance structures.

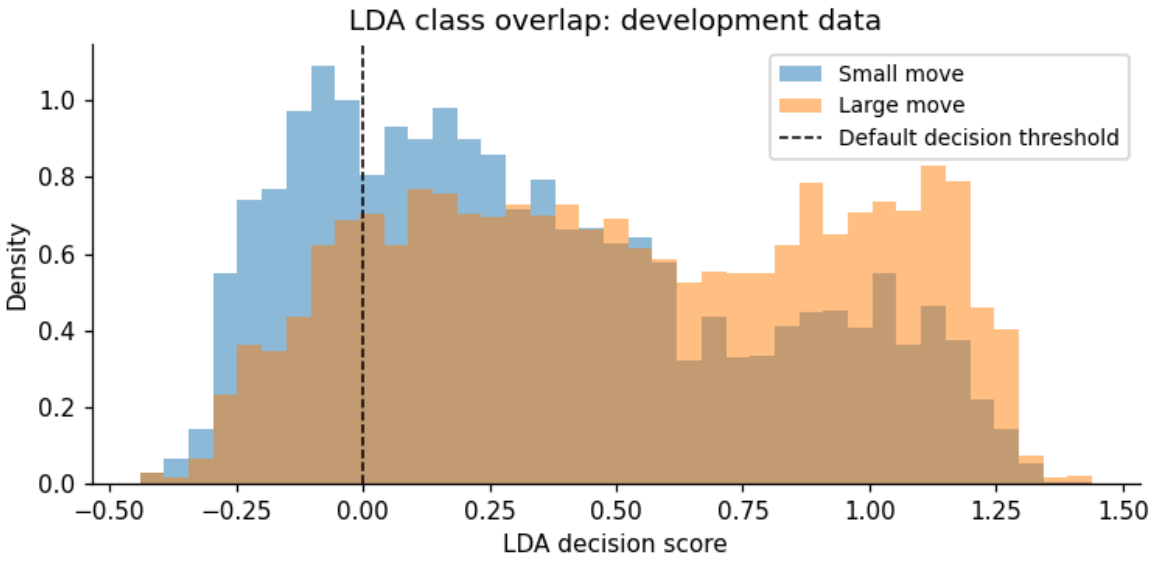
We compare the ordinary covariance estimate with automatic shrinkage, using the `lsqr` solver and training-sample class priors. The score distributions below are from the fitted development sample. They help show class overlap, but are not an out-of-sample accuracy result.

In [10]:
```
lda = run_search(
    Pipeline([('scale', StandardScaler()),
              ('model', LinearDiscriminantAnalysis(solver='lsqr'))]),
    {'model__shrinkage': [None, 'auto']}, c_dev, 'roc_auc')
show_search(lda)
lda_scores = lda.best_estimator_.decision_function(X_dev)
fig, ax = plt.subplots(figsize=(7, 3.5))
bins = np.linspace(lda_scores.min(), lda_scores.max(), 40)
for label, name in [(0, 'Small move'), (1, 'Large move')]:
    ax.hist(lda_scores[c_dev.to_numpy() == label], bins=bins, density=Tru
            alpha=0.5, label=name)
ax.axvline(0, color='black', lw=1, ls='--', label='Default decision thres
ax.set(xlabel='LDA decision score', ylabel='Density', title='LDA class ov
ax.legend(fontsize=9)
```

```
fig.tight_layout()
plt.show()
```

	params	Mean CV score	SD across folds
0	{'model__shrinkage': None}	0.6163	0.0008
1	{'model__shrinkage': 'auto'}	0.6163	0.0008

```
Selected: {'model__shrinkage': None}
```



The ordinary covariance estimate is selected, but both settings round to the same mean validation AUC of 0.6163. We would not read much into such a small difference. The score histograms overlap substantially, so the linear score separates the groups only partly. LDA still provides a useful simple comparison: a more flexible classifier should add something beyond this level of separation to justify its extra complexity.

8. Support Vector Machine — category 6

An SVM looks for a wide separating margin while allowing some observations to fall inside that margin or on the wrong side of the boundary. The parameter C controls the penalty for these margin violations. We use the radial basis function (RBF) kernel,

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2),$$

which allows a nonlinear boundary. The inputs are the scaled features and class labels. Outputs are a decision score and a class prediction. We leave probability calibration off: the score is suitable for AUC, but is not a probability.

We tune C and γ on the same time folds. A flexible boundary may help if the relationship is nonlinear, but it can also fit local noise and is harder to explain than a tree. Scaling matters because the RBF kernel depends on distance. The plot varies P/B and volatility while fixing the other four inputs at their development medians; it is one slice of a six-dimensional decision function.

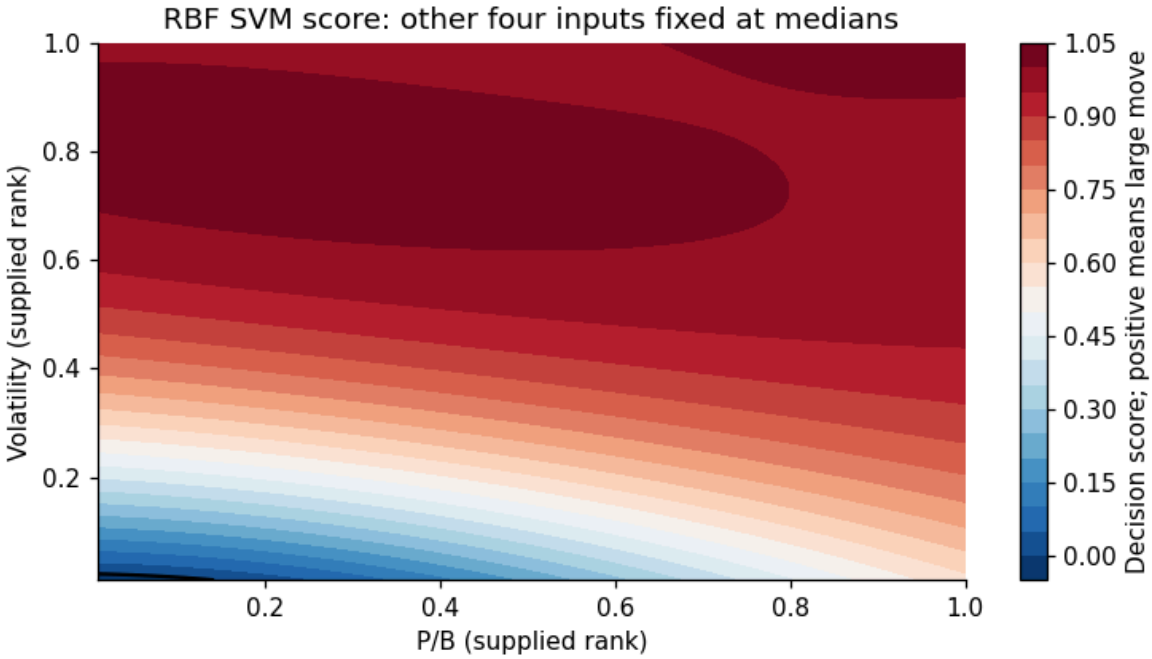
In [11]:
```
svm = run_search(
    Pipeline([('scale', StandardScaler()),
```

```
                ('model', SVC(kernel='rbf'))]),
    {'model__C': [0.1, 1, 10], 'model__gamma': ['scale', 0.1]},
    c_dev, 'roc_auc')
show_search(svm)

horizontal = np.linspace(X_dev.Pb.min(), X_dev.Pb.max(), 90)
vertical = np.linspace(X_dev.Vol1Y_Usd.min(), X_dev.Vol1Y_Usd.max(), 90)
xx, yy = np.meshgrid(horizontal, vertical)
slice_data = pd.DataFrame(np.tile(X_dev.median().to_numpy(), (xx.size, 1)
slice_data['Pb'], slice_data['Vol1Y_Usd'] = xx.ravel(), yy.ravel()
surface = svm.best_estimator_.decision_function(slice_data).reshape(xx.sh
fig, ax = plt.subplots(figsize=(7, 4))
im = ax.contourf(xx, yy, surface, levels=20, cmap='RdBu_r')
if surface.min() < 0 < surface.max():
    ax.contour(xx, yy, surface, levels=[0], colors='black', linewidths=1.
ax.set(xlabel='P/B (supplied rank)', ylabel='Volatility (supplied rank)',
       title='RBF SVM score: other four inputs fixed at medians')
fig.colorbar(im, ax=ax, label='Decision score; positive means large move'
fig.tight_layout()
plt.show()
```

	params	Mean CV score	SD across folds
0	{'model__C': 0.1, 'model__gamma': 0.1}	0.5838	0.0063
1	{'model__C': 0.1, 'model__gamma': 'scale'}	0.5819	0.0090
2	{'model__C': 1, 'model__gamma': 0.1}	0.5800	0.0069
3	{'model__C': 1, 'model__gamma': 'scale'}	0.5767	0.0091
4	{'model__C': 10, 'model__gamma': 0.1}	0.5735	0.0103
5	{'model__C': 10, 'model__gamma': 'scale'}	0.5680	0.0098

```
Selected: {'model__C': 0.1, 'model__gamma': 0.1}
```



The selected values are $C = 0.1$ and $\gamma = 0.1$, with mean validation AUC of 0.5838. Larger values of C do not help within this grid. Most of the plotted slice has a positive score, especially at higher volatility ranks, so it mostly predicts a large move

there. The contour describes inputs with four features fixed at their medians; it is not a map of every observed stock. The scores should not be read as calibrated event probabilities.

9. Neural network — category 7

We use a small feedforward network with one hidden layer:

$$h = \mathrm{ReLU}(W_1 x + b_1), \qquad \hat{p} = \sigma(w_2^\top h + b_2).$$

It receives the six scaled inputs and returns an estimated probability of a large move. The weights are fitted by backpropagation, using cross-entropy loss with an L2 penalty. We compare 8 and 16 hidden units and two penalty values. ReLU and Adam are fixed, as are the seed, batch size, learning rate, and iteration limit.

The implementation uses scikit-learn's `MLPClassifier`, which is a dense neural network like the one introduced in Module 5. It avoids needing a separate deep-learning package for this small example. A network can learn nonlinear combinations of features, but its result depends on optimization and is less interpretable. We disable its random internal validation split (`early_stopping=False`) and use the shared time folds for model selection. We also check that training converges within the iteration limit. MLP documentation.

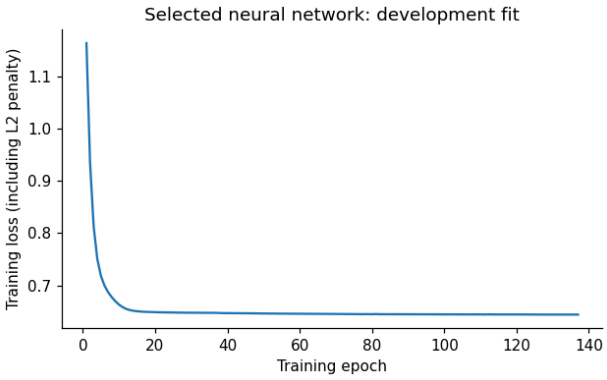
In [12]:
```
network = run_search(
    Pipeline([('scale', StandardScaler()),
              ('model', MLPClassifier(activation='relu', solver='adam',
                                      batch_size=128, learning_rate_init=
                                      max_iter=600, tol=1e-4, n_iter_no_c
                                      early_stopping=False, random_state=
    {'model__hidden_layer_sizes': [(8,), (16,)], 'model__alpha': [0.001,
    c_dev, 'roc_auc')
show_search(network)
mlp = network.best_estimator_.named_steps['model']
print('Iterations used:', mlp.n_iter_)
print('Trainable parameters:', sum(w.size for w in mlp.coefs_) + sum(b.si

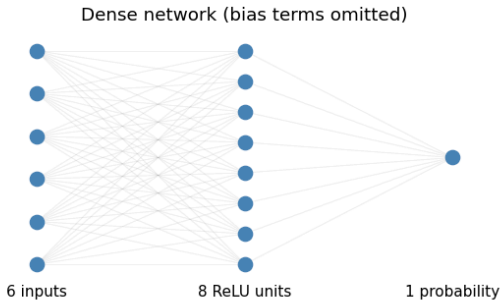
fig, axes = plt.subplots(1, 2, figsize=(11, 3.7))
axes[0].plot(np.arange(1, len(mlp.loss_curve_) + 1), mlp.loss_curve_)
axes[0].set(xlabel='Training epoch', ylabel='Training loss (including L2
            title='Selected neural network: development fit')
widths = [6, mlp.hidden_layer_sizes[0], 1]
positions = [np.linspace(-1, 1, w) if w > 1 else np.array([0.0]) for w in
for layer in range(2):
    for left in positions[layer]:
        for right in positions[layer + 1]:
            axes[1].plot([layer, layer + 1], [left, right], color='gray',
for layer, ys in enumerate(positions):
    axes[1].scatter(np.repeat(layer, len(ys)), ys, s=85, color='steelblue
for layer, label in enumerate(['6 inputs', f'{widths[1]} ReLU units', '1
    axes[1].text(layer, -1.3, label, ha='center')
axes[1].set_title('Dense network (bias terms omitted)')
axes[1].set(xlim=(-0.3, 2.3), ylim=(-1.6, 1.2))
axes[1].axis('off')
```

```
fig.tight_layout()
plt.show()
```

	params	Mean CV score	SD across folds
0	{'model__alpha': 0.001, 'model__hidden_layer_sizes': (8,)}	0.6128	0.0044
1	{'model__alpha': 0.01, 'model__hidden_layer_sizes': (8,)}	0.6120	0.0042
2	{'model__alpha': 0.001, 'model__hidden_layer_sizes': (16,)}	0.6119	0.0072
3	{'model__alpha': 0.01, 'model__hidden_layer_sizes': (16,)}	0.6114	0.0072

```
Selected: {'model__alpha': 0.001, 'model__hidden_layer_sizes': (8,)}
Iterations used: 137
Trainable parameters: 65
```





The selected network has 8 hidden units and an L2 penalty of 0.001. It has only 65 trainable parameters and a mean validation AUC of 0.6128. The development fit stops after 137 epochs, within the 600-epoch limit, and its loss curve flattens early. A flat training curve tells us that further optimization has slowed; it does not establish that the model will generalize. Increasing the hidden layer to 16 units did not improve the mean validation score in this search.

10. Holdout results: 2015–2018

All choices above are now fixed. Each selected model has been refitted on the full development period, and the fitted models stay unchanged throughout the holdout. The first baseline predicts the development mean return. The classification baseline predicts the development majority class and assigns a constant class probability.

The two tasks need separate tables. For returns, we report root mean squared error (RMSE), mean absolute error (MAE), and R^2. RMSE and MAE are in percentage points of monthly return. For classification, we report ROC AUC, balanced accuracy, ordinary accuracy, and F1 for the large-move class. The majority baseline matters because the classes are not evenly balanced. No decision threshold is tuned on the test set.

In [13]:
```
mean_baseline = DummyRegressor(strategy='mean').fit(X_dev, y_dev)
return_predictions = {
    'Mean baseline': mean_baseline.predict(X_test),
    'Ridge': ridge.best_estimator_.predict(X_test),
}
regression_results = pd.DataFrame([
    {'Model': name, 'RMSE (pp)': 100 * root_mean_squared_error(y_test, pr
     'MAE (pp)': 100 * mean_absolute_error(y_test, prediction),
     'R2': r2_score(y_test, prediction)}
    for name, prediction in return_predictions.items()
]).set_index('Model')
print('Return prediction')
display(regression_results)

prior_baseline = DummyClassifier(strategy='prior').fit(X_dev, c_dev)
classifiers = {'Majority baseline': prior_baseline,
               'Tree': tree.best_estimator_, 'LDA': lda.best_estimator_,
               'RBF SVM': svm.best_estimator_, 'Neural network': network.
class_predictions, class_scores, classification_rows = {}, {}, []
for name, estimator in classifiers.items():
    prediction = estimator.predict(X_test)
    score = (estimator.decision_function(X_test) if hasattr(estimator, 'd
             else estimator.predict_proba(X_test)[:, 1])
    class_predictions[name], class_scores[name] = prediction, score
    classification_rows.append({
        'Model': name, 'ROC AUC': roc_auc_score(c_test, score),
        'Balanced accuracy': balanced_accuracy_score(c_test, prediction),
        'Accuracy': accuracy_score(c_test, prediction),
        'F1 (large move)': f1_score(c_test, prediction, zero_division=0),
    })
classification_results = pd.DataFrame(classification_rows).set_index('Mod
print(f'Large-move classification; holdout class 1 share = {c_test.mean()
display(classification_results)

assert all(len(pred) == len(test) for pred in return_predictions.values()
assert all(np.isfinite(score).all() for score in class_scores.values())
assert np.isclose(classification_results.loc['Majority baseline', 'ROC AU
```

```
Return prediction
```

Model	RMSE (pp)	MAE (pp)	R2
Mean baseline	10.0027	6.9375	-0.0021
Ridge	10.0095	6.9686	-0.0035

```
Large-move classification; holdout class 1 share = 57.83%
```

Model	ROC AUC	Balanced accuracy	Accuracy	F1 (large move)
Majority baseline	0.5000	0.5000	0.5783	0.7328
Tree	0.5982	0.5421	0.5934	0.7122
LDA	0.6079	0.5478	0.5964	0.7109
RBF SVM	0.5817	0.5065	0.5804	0.7294
Neural network	0.6050	0.5450	0.5976	0.7170

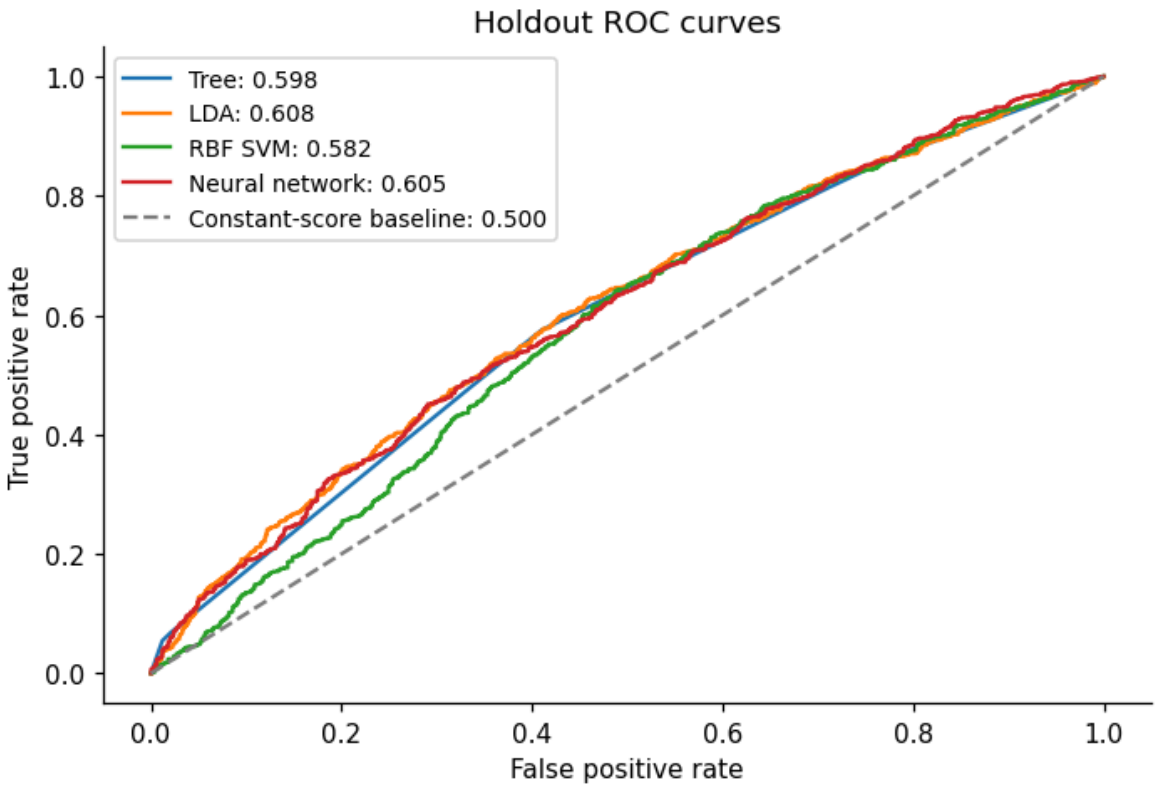
Ridge does not improve on the mean-return baseline. Its holdout RMSE is 10.0095 percentage points, compared with 10.0027 for the baseline; its MAE is also slightly higher. Both R^2 values are just below zero. In this Ridge specification, the six features do not improve on the constant-return forecast.

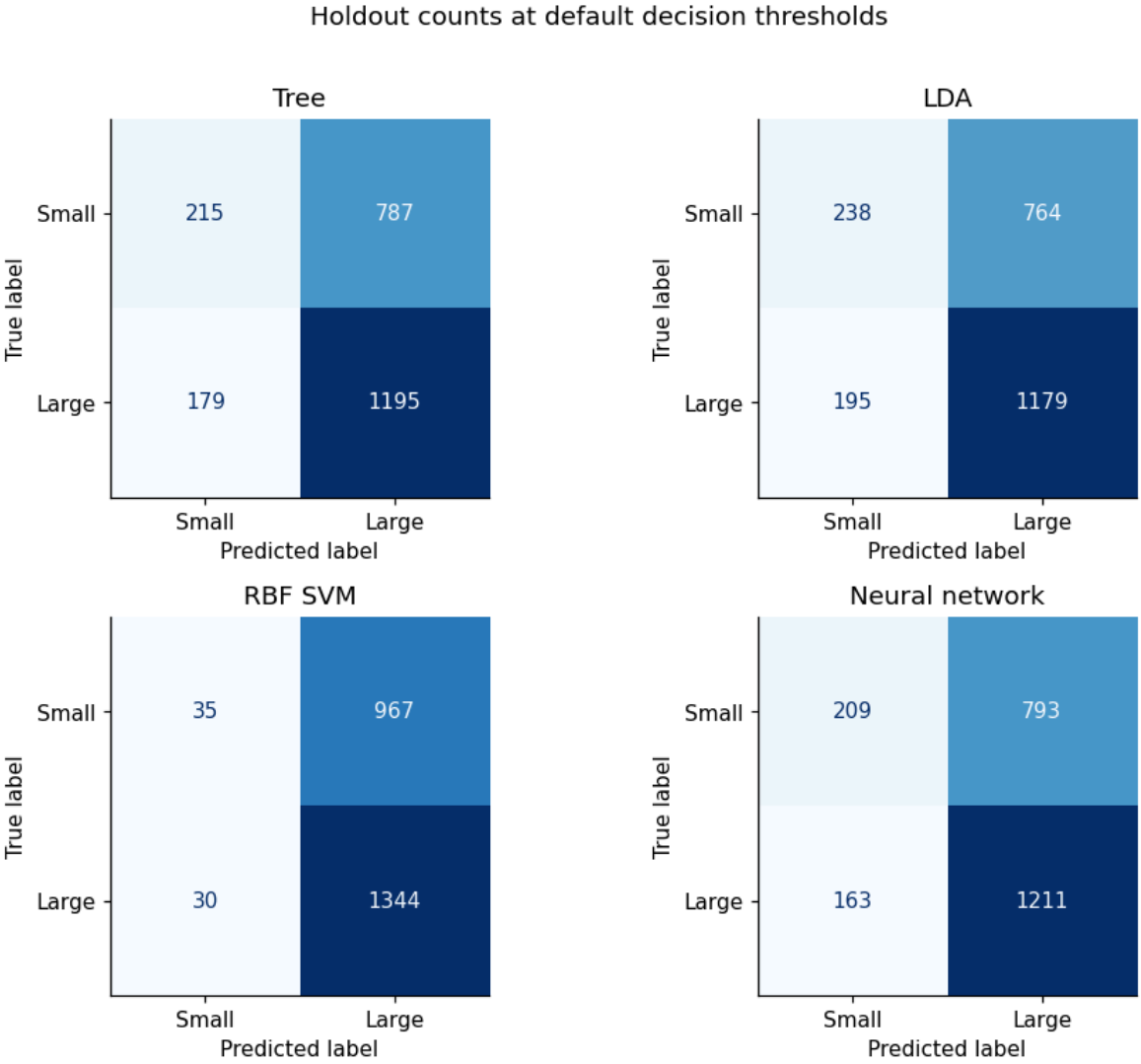
For large-move classification, LDA has the highest AUC at 0.6079, followed closely by the neural network at 0.6050. Their balanced accuracies are only about 0.548 and 0.545. The network has the highest ordinary accuracy, 0.5976, but that is still only about two percentage points above the majority baseline. These differences are small, and we have not tested their statistical significance.

The majority baseline reaches the highest F1 score, 0.7328, by predicting every row as a large move. This gives it recall of 1 for that class, while detecting no small moves. The confusion matrices show why F1 alone is insufficient here: identifying small moves is difficult for all four classifiers. The SVM gets only 35 of the 1,002 small moves right, even though its AUC is above 0.5. Ranking scores and default-threshold classifications answer different questions.

In [14]:
```
fig, ax = plt.subplots(figsize=(6.5, 4.5))
for name in list(classifiers)[1:]:
    fpr, tpr, _ = roc_curve(c_test, class_scores[name])
    ax.plot(fpr, tpr, label=f"{name}: {classification_results.loc[name, '
ax.plot([0, 1], [0, 1], color='gray', ls='--', label='Constant-score base
ax.set(xlabel='False positive rate', ylabel='True positive rate', title='
ax.legend(fontsize=9)
fig.tight_layout()
plt.show()

fig, axes = plt.subplots(2, 2, figsize=(9, 7))
for ax, name in zip(axes.ravel(), list(classifiers)[1:]):
    ConfusionMatrixDisplay.from_predictions(
        c_test, class_predictions[name], labels=[0, 1],
        display_labels=['Small', 'Large'], colorbar=False, ax=ax, cmap='B
    ax.set_title(name)
fig.suptitle('Holdout counts at default decision thresholds', y=1.01)
fig.tight_layout()
plt.show()
```





Monthly observations from different stocks are not independent. They share market conditions, and the same stocks appear repeatedly. A pooled score can therefore

hide differences between years. The table below breaks down the frozen predictions by year. It is a descriptive check, not a second round of model selection or a test of statistical significance.

In [15]:
```
year_rows = []
for year in sorted(test.date.dt.year.unique()):
    mask = (test.date.dt.year == year).to_numpy()
    row = {'Year': year, 'Observations': int(mask.sum()),
           'Large-move share': float(c_test[mask].mean())}
    for name in list(classifiers)[1:]:
        row[name] = roc_auc_score(c_test[mask], class_scores[name][mask])
    year_rows.append(row)
yearly_auc = pd.DataFrame(year_rows).set_index('Year')
display(yearly_auc)
```

Year	Observations	Large-move share	Tree	LDA	RBF SVM	Neural network
2015	599	0.5960	0.6171	0.6237	0.6158	0.6304
2016	595	0.6050	0.5961	0.5906	0.5541	0.5871
2017	600	0.5050	0.6199	0.6421	0.5995	0.6361
2018	582	0.6082	0.5666	0.5751	0.5623	0.5660

The neural network has the highest AUC in 2015 (0.6304), the tree in 2016 (0.5961), and LDA in 2017 and 2018. LDA ranges from 0.5751 in 2018 to 0.6421 in 2017. All four models perform worse in 2018 than in 2017 by this measure. This variation makes the pooled ranking less convincing as evidence of a lasting advantage, especially with only four holdout years.

11. What we take from the comparison

Method	Main output in this notebook	Useful aspect	Main limitation here
Ridge	Next-month return estimate	Small, interpretable linear model	Cannot represent nonlinear relationships
K-means	Stock-month cluster and feature centers	Describes recurring feature profiles	Distance and K determine the grouping
PCA	Component scores and variance shares	Summarizes correlated inputs	High variance need not predict returns
Classification tree	Large-move probability and class	Visible thresholds and interactions	Splits can change with the sample
LDA	Linear score, probability, and class	Simple comparison with few tuning choices	Shared Gaussian covariance assumptions are restrictive
RBF SVM	Nonlinear decision score and class	Flexible boundary using distances	Harder to explain; scores are not probabilities

Method	Main output in this notebook	Useful aspect	Main limitation here
Neural network	Large-move probability and class	Learns nonlinear feature combinations	Training and interpretation require more care

Clustering, dimension reduction, return estimation, and classification solve different problems. It would not make sense to rank all seven using one performance number. None of these examples establishes a profitable strategy: a large-move label does not provide a direction, and the notebook does not model trading costs or positions.

The main result for us is that adding flexibility does not automatically improve the forecast. LDA's holdout AUC is close to the small network's, even though it uses a simpler decision boundary. The RBF SVM has a weaker ranking score and predicts the majority class most of the time. Ridge shows how to regularize a return model, but its test result gives no reason to prefer it over the mean baseline here.

K-means and PCA are more helpful for describing the inputs: both highlight the relationship between profitability and valuation ranks. We would use those results to understand the sample, rather than present them as evidence of alpha. The seven methods are useful in different ways, and the baselines make their limits easier to see.

12. Limitations and sources

The sample is small in terms of companies, and the stock IDs are anonymous. We cannot verify the original universe selection, delisting treatment, or when every accounting input became available. A chronological split helps with the evaluation order, but it does not establish that the source features were available to an investor at the date shown. We keep extreme returns rather than deciding retrospectively which ones to remove.

The feature set, 4% cutoff, and search grids are deliberately limited. The network is fitted with one seed, and the classifiers are not probability-calibrated. We have not estimated uncertainty with a time-block bootstrap. A useful next exercise would be to check the frozen setup on a separate period or dataset, then study a direction-specific target with realistic publication lags and costs.

Course material used: WorldQuant University, *MScFE 632 Machine Learning in Finance*, GWP1 brief (2026); Module 1 regularization; Module 2 clustering and PCA; Module 3 classification trees and Bayesian classification; Module 5 support vector machines and neural networks. These supplied lessons guide the methods, the six features, and the absolute-return label. The code, time splits, and comparisons in this notebook are assembled for this experiment.

Data and implementation references (accessed 11 Oct. 2026):

1. Coqueret, Guillaume, and Tony Guida. “Notations and Data.” *Machine Learning for Factor Investing*. mlfactor.com/notdata.html.
2. Coqueret, Guillaume, and Tony Guida. “Data Description.” *Machine Learning for Factor Investing*. mlfactor.com/data-description.html.
3. scikit-learn developers. “TimeSeriesSplit.” *scikit-learn Documentation*. API reference.
4. scikit-learn developers. “Neural Network Models (Supervised).” *scikit-learn User Guide*. User guide.